

Tempus Core: Area-Power Efficient Temporal-Unary Convolution Core for Low-Precision Edge DLAs

Prabhu Vellaisamy, Harideep Nair, Thomas Kang, Yichen Ni, Haoyang Fan, Bin Qi, Jeff Chen, Shawn Blanton, and John Paul Shen

ECE Department, [Carnegie Mellon University](#)

{pvellais, hpnair, thomaska, yichenn, hfan2, binq, jhchen2, rblanton, jpshen}@andrew.cmu.edu

Abstract—The increasing complexity of deep neural networks (DNNs) poses significant challenges for edge inference deployment due to resource and power constraints of edge devices. Recent works on unary-based matrix multiplication hardware aim to leverage data sparsity and low-precision values to enhance hardware efficiency. However, the adoption and integration of such unary hardware into commercial deep learning accelerators (DLA) remain limited due to processing element (PE) array dataflow differences. This work presents *Tempus Core*, a convolution core with highly scalable unary-based PE array comprising of *tub* (temporal-unary-binary) multipliers that seamlessly integrates with the NVDLA (NVIDIA’s open-source DLA for accelerating CNNs) while maintaining dataflow compliance and boosting hardware efficiency. Analysis across various datapath granularities shows that for INT8 precision in 45nm CMOS, Tempus Core’s PE cell unit (PCU) yields 59.3% and 15.3% reductions in area and power consumption, respectively, over NVDLA’s CMAC unit. Considering a 16x16 PE array in Tempus Core, area and power improves by 75% and 62%, respectively, while delivering 5x and 4x iso-area throughput improvements for INT8 and INT4 precisions. Post-place and route analysis of Tempus Core’s PCU shows that the 16x4 PE array for INT4 precision in 45nm CMOS requires only 0.017mm² die area and consumes only 6.2mW of total power. We demonstrate that area-power efficient unary-based hardware can be seamlessly integrated into conventional DLAs, paving the path for efficient unary hardware for edge AI inference.

Index Terms—NVDLA, GEMM, Temporal-Unary, Convolution MAC core, Low Precision, CNNs

I. INTRODUCTION

Since the introduction of AlexNet [1] in 2012, AI computational demands have grown rapidly [2], driving the adoption of hardware accelerators like GPUs and TPUs for deep learning (DL) training and inference. However, DL’s computational needs have quickly outpaced the growth of available compute power, leading to what is now termed the “AI compute gap”. This gap has renewed interest in innovative computational techniques and architectures aimed at closing this compute performance and energy efficiency gap.

In recent years, quantization techniques have become an effective strategy for deploying AI models with reduced model complexity. While 32-bit floating point (FP32) was the standard for DL training, FP16 is now widely adopted, offering 4x-8x performance improvements [3]. Building on this trend, FP8 precision for both training and inference has been proposed [4] and implemented by Nvidia in their H100 GPUs, which support FP8 models through the Transformer

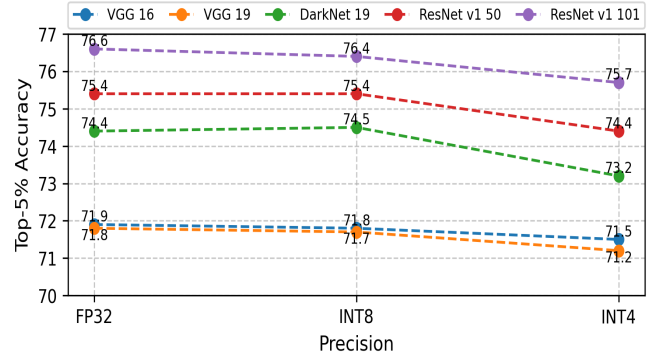


Fig. 1. Quantization training accuracies achieved on different ImageNet CNNs for different integer-based precisions when compared to baseline FP32 precision [8]. Results show minimal accuracy decrease with lower precisions.

Engine library [5]. In particular, FP8 training on various convolutional neural network (CNN) models has shown minimal accuracy degradation while increasing throughput by 2x-4x [3]. In addition to floating point formats, 8-bit integer (INT8) training has been demonstrated to reduce the training time of CNNs on Pascal GPUs by 22% [6]. Although INT8 has become the standard for DL inference, a shift toward INT4 precision has led to a 77% performance improvement over INT8 [7]. The results of ImageNet trainings for quantized CNN models and their corresponding Top-5% accuracy are presented in Fig. 1, showing minimal accuracy degradation for INT4 quantized models compared to their baseline FP32 implementations. These quantization results offer tremendous promise particularly for edge inference deployment, where computational resources are limited.

Unary computing has recently emerged as a promising paradigm for low-precision AI, enabling highly area- and power-efficient hardware by trading off latency for efficiency. In particular, temporal encoding techniques have been shown to significantly reduce computational overhead in matrix multiplication units, as demonstrated by recent works [9] [10] [11], which exploit the inherent sparsity of DL models. However, these existing implementations focus primarily on GEMM-level designs and are not scaled to inherently support convolution dataflows. This work addresses that gap by extending unary computing to convolution dataflows, while ensuring compatibility with widely-used deep learning accelerators